

Project 3

```
# Importing additional libraries
library(lubridate)
library(stats)
library(broom)
library(ggforce)
# Setting a seed for reproducibility
set.seed(116)
```

Question:

Do Netflix movies form distinct performance segments based on viewer engagement and viewing efficiency, and does global availability of movies affect these segments?

Introduction

This project analyzes movie viewing data from Netflix's Engagement Reports, from July 2023 to June 2025. The dataset used in this project is publicly available on the [TidyTuesday GitHub repository](#).

For the purpose of this project, the analysis focuses exclusively on the `Movies` dataset. The dataset consists of approximately 36,000 observations and 8 variables. It includes information such as the source of the collected data, movie title, global availability status, and key performance metrics such as - the number of views, total hours viewed, movie duration, and release date.

To answer this question, we will use the `runtime` (movie duration in HMS (string)), `views` (number of movie views (integer)) and `available_globally` (Yes or No (binary)) columns from the dataset, since we will have to segment the movies based on viewership and engagement. "Viewer Engagement" can be inferred from the movie's runtime in relation to its total number of views. A more detailed explanation on the methodology is provided in the Approach section.

Approach

The question consists of two main components:

- 1) Segmenting movies based on viewership and engagement performance
- 2) Examining the effect of global availability on the performance of each segment

To address the *first part of the question*, we will focus on the `runtime` and `views` variables. We will begin with data cleaning, where observations with missing values will be removed. Next, additional variables will be created to better capture the movie performance:

- **total_runtime_mins**: the total movie runtime in minutes.
- **log_views**: a log-transformed version of the views variable.

- **log_efficiency**: a log-transformed efficiency measure (calculated as $\text{views} / \text{total_runtime_mins}$). This metric measures the number of views relative to the length of the movie.

The final dataset used for clustering will therefore contain three variables: `total_runtime_mins`, `log_views`, and `log_efficiency`. The first two variables represents the overall engagement, whereas `log_efficiency` represents the viewing performance efficiency relative to movie length.

The next step would be finding the optimal number of clusters using the Elbow method to later perform K-Means Clustering.

To interpret the resulting clusters, a **compound plot** combining a scatterplot, sina plot overlayed by a violin plot and box plot will be created. We will be creating the following visualizations :

- **Scatter plot** : to visualize all the clusters formed by the k-means clustering algorithm. This visualization helps in identifying the separation, overlap, and overall structure of the clusters, making it easier to assess whether the algorithm has successfully identified distinct performance segments.
- **Sina plot overlayed by a violing and box plot**: to compare the differences in the views between the clusters. This plot will provide a comprehensive view of the distribution of views within each cluster. The sina plot shows how the data points are spread within each cluster. The violin plot highlights the overall distribution and density shape. Finally, the box plot provides a 5 number summary, allowing for direct comparison of the central tendency and spread of views across the clusters.
- **Box plot** : to compare the difference in the efficiency between the clusters. This visualization provides an effective way to compare the median and variability among the clusters.

We will be using **color mapping** to ensure that a consistent color coding is maintained across all plots to enable clear comparisons between clusters.

For the *second part of the analysis*, the cluster labels generated from the K-means model will be merged back with the cleaned and scaled dataset. The data will then be grouped by cluster and global availability status.

Within each cluster, both the count and percentage of globally available movies will be calculated. To visualize these proportions, a 100% stacked bar chart will be used. This type of chart allows for a direct comparison of the relative share of globally available v/s non-globally available movies within each cluster, regardless of differences in cluster size. We will use this plot since we are comparing proportions instead of absolute counts across the clusters. **Color mapping** will be applied to the bars based on the global availability.

Analysis

```
# ===== Data Loading =====  
# Reading movies dataset directly from github repo  
movies <- readr::read_csv('https://raw.githubusercontent.com/rfordatascience/  
tidytuesday/main/data/2025/2025-07-29/movies.csv')
```

```
# ===== Getting to know the data =====  
glimpse(movies)
```

```
Rows: 36,121  
Columns: 8  
$ source      <chr>  
"1_What_We_Watched_A_Netflix_Engagement_Report_2025..."  
$ report      <chr> "2025Jan-Jun", "2025Jan-Jun", "2025Jan-Jun",  
"2025J..."  
$ title       <chr> "Back in Action", "STRAW", "The Life List",  
"Exterr..."  
$ available_globally <chr> "Yes", "Yes", "Yes", "Yes", "Yes", "No", "Yes",  
"Ye..."  
$ release_date <date> 2025-01-17, 2025-06-06, 2025-03-28, 2025-04-30,  
20...  
$ hours_viewed <dbl> 313000000, 185200000, 198900000, 159000000,  
1549000...  
$ runtime     <chr> "1H 54M 0S", "1H 48M 0S", "2H 5M 0S", "1H 49M 0S",  
...  
$ views       <dbl> 164700000, 102900000, 95500000, 87500000,  
86900000,...
```

```
# ===== Data Cleaning and Wrangling =====  
  
# Creating a subset of the movies table  
sub_movies1 <- movies |>  
  select(runtime, views) |> # choosing only runtime and views columns  
  filter(!is.na(runtime), !is.na(views)) |> # filtering out rows with missing  
  values  
  mutate(  
    # Extracting hours from runtime and replace NA with 0  
    runtime_hr = as.numeric(str_extract(runtime, "\\d+(?=H)")),  
    runtime_hr = replace_na(runtime_hr, 0),  
  
    # Extracting minutes from runtime and replace NA with 0  
    runtime_min = as.numeric(str_extract(runtime, "\\d+(?=M)")),  
    runtime_min = replace_na(runtime_min, 0),  
  
    # Calculating the total runtime in minutes
```

```

total_runtime_mins = (runtime_hr * 60) + runtime_min,

# applying log transformation to views column
# Using views + 1 to prevent -Inf in case the views are 0
log_views = log10(views + 1),

# metric to measure performance efficiency (The other 2 variables capture
engagement volume)
log_efficiency = log10((views / total_runtime_mins) + 1)
) |>
select(total_runtime_mins, log_views, log_efficiency)

```

```

# Removing the outliers from the dataset using IQR

# Calculating the upper and lower quantile and IQR
Q1 <- quantile(sub_movies1$total_runtime_mins, 0.25)
Q3 <- quantile(sub_movies1$total_runtime_mins, 0.75)

IQR_val <- IQR(sub_movies1$total_runtime_mins)

upper_bound <- Q3 + 1.5 * IQR_val # 149
lower_bound <- Q1 - 1.5 * IQR_val # 61

# I am widening the range a little based on the bounds calculated using IQR
upper_bound = 210
lower_bound = 20

# Filtering out rows that exceed this range
sub_movies_out1 <- sub_movies1 |>
  filter(
    (total_runtime_mins <= upper_bound)
    &
    (total_runtime_mins >= lower_bound)
  )

# scaling the resulting dataset
scaled_data1 <- scale(sub_movies_out1)
scaled_data1 <- as.data.frame(scale(sub_movies_out1))

```

```

# ===== Picking K (Elbow Method) =====

# Elbow method to pick the optimal number of clusters
elbow_method <- function(data, centers) {
  km_fit <- kmeans(data, centers = centers, iter.max = 50, nstart = 10)
  return(km_fit$tot.withinss)
}

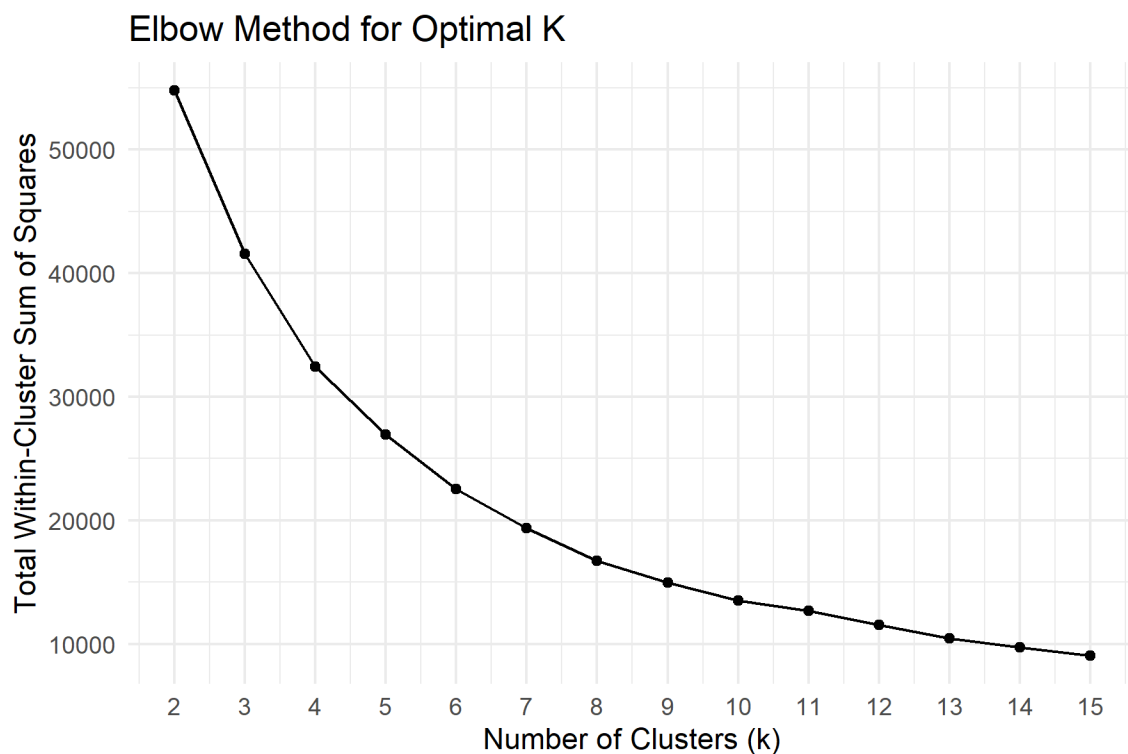
```

```

# Run the elbow analysis
elbow_results <- tibble(centers = 2:15) |> # Testing cluster number from 2 to
15
  mutate(
    within_sum_squares = map_dbl(centers, ~elbow_method(scaled_data1, .x))
  )

# Plotting
ggplot(elbow_results, aes(x = centers, y = within_sum_squares)) +
  geom_point() +
  geom_line() +
  scale_x_continuous(breaks = 2:15) +
  labs(
    title = "Elbow Method for Optimal K",
    x = "Number of Clusters (k)",
    y = "Total Within-Cluster Sum of Squares"
  ) +
  theme_minimal()

```



```
# ===== K-Means Clustering =====
set.seed(116)
km_fit_finall <- scaled_data1 |>
  kmeans(
    centers = 4, # number of centroids (obtained from the elbow method)
    nstart = 10, # number of restarts of the algorithm
    iter.max = 50
  )
```

```
# Joining the clusters to the original data (including other columns as well)
cleaned_movies1 <- movies |>
  filter(!is.na(runtime), !is.na(views)) |>
  mutate(
    # Extract hours and replace NA with 0
    runtime_hr = as.numeric(str_extract(runtime, "\\d+(?=H)")),
    runtime_hr = replace_na(runtime_hr, 0),

    # Extract minutes and replace NA with 0
    runtime_min = as.numeric(str_extract(runtime, "\\d+(?=M)")),
    runtime_min = replace_na(runtime_min, 0),

    # Calculating the total runtime in minutes
    total_runtime_mins = (runtime_hr * 60) + runtime_min,

    # Using views + 1 to prevent -Inf values if views are 0
    log_views = log10(views + 1),
    log_efficiency = log10((views / total_runtime_mins) + 1)
  ) |>
  select(-runtime_hr, -runtime_min, -source)

# Filtering out the outliers from data
cleaned_movies1 <- cleaned_movies1 |>
  filter(
    (total_runtime_mins <= upper_bound) &
    (total_runtime_mins >= lower_bound)
  )
```

```
# Combining the cluster assignments with the cleaned (unscaled) dataset
combined <- km_fit_finall |>
  augment(cleaned_movies1)

# Mapping the cluster names to the clusters
combined <- combined |>
  mutate(
    cluster_names = case_match(
      .cluster,
```

```

    "2" ~ "Cluster A",
    "4" ~ "Cluster B",
    "1" ~ "Cluster C",
    "3" ~ "Cluster D"
  ),
  cluster_names = factor(
    cluster_names,
    levels = c("Cluster A", "Cluster B", "Cluster C", "Cluster D")
  )
)

# Mapping cluster names to the cluster numbers
#   "2" ~ "Cluster A: High-performing",
#   "4" ~ "Cluster B: Short-form Moderate Performers",
#   "1" ~ "Cluster C: Long-form Moderate Performers",
#   "3" ~ "Cluster D: Low-performing"

#|
# ===== Visualization 1 =====

custom_colors <- c("#F8766D", "#7CAE00", "#00BFC4", "#C77CFF") # custom color
pallet for color mapping

# Scatter plot to visualize the clusters formed by K-means algorithm
p1 <- combined |>
  ggplot(aes(x = total_runtime_mins, y = log_efficiency)) +
  geom_point(aes(color = cluster_names), alpha = 0.4) +
  labs(
    title = "Viewing Efficiency vs Movie Runtime",
    x = "Movie Runtime (mins)",
    y = "Efficiency (Log transformed)"
  ) +
  scale_color_manual(values = custom_colors) +
  theme_minimal() +
  theme(legend.position = "top")

# Box plot to plot how the viewing efficiency of movies differs across the
clusters
p2 <- combined |>
  ggplot(aes(cluster_names, log_efficiency, fill = cluster_names)) +
  geom_boxplot() +
  labs(
    title = "Viewing Efficiency by Cluster",
    x = NULL,
    y = "Efficiency (Log transformed)",
  ) +
  scale_fill_manual(values = custom_colors) +

```

```

theme_minimal() +
theme(legend.position = "none")

# Sina plot overlaid by a violin and box plot
p3 <- combined |>
  ggplot(aes(cluster_names, log_views)) +
  geom_sina(aes(color = cluster_names), maxwidth=1) +
  geom_violin(alpha = 0.1, width = 1, color = "black", fill=NA) + # only
  outlining the shape
  geom_boxplot(alpha = 0.6) + # reducing the opacity
  labs(title = "Distribution of Log Views by Cluster",
        x= NULL,
        y= "Views (Log transformed)",
        caption = "Cluster A: High-performing\nCluster B: Short-form Moderate
Performers\nCluster C: Long-form Moderate Performers\nCluster D: Low-
performing"
        ) +
  scale_color_manual(values = custom_colors) +
  theme_minimal() +
  theme(
    legend.position = "none",
    # Adding a caption to indicate what each cluster represents
    plot.caption = element_text(hjust = 0, face = "italic", size = 9)
  )

# Combining the 3 plots (Compound Plot)
(p1 | p2) / p3

```




**** NOTE:** In plot showing the distribution of Log Views by Cluster plot, we can see that there is no box around cluster D. This is because, the variance among the datapoints in this cluster is extremely low. If we look at the summary statistics below, the median and the minimum values are identical, hence, the lower end of the box and the median line are the same. We can also see that nearly all the values are clumped right at the log_views value of 5 (keep in mind that this cluster has the most number of datapoints).

```
# ===== Getting the summary statistics of the clustered data
=====
# Join the cluster assignments to your unscaled data
cluster_comparison <- combined |>
```

```

group_by(cluster_names) |>
summarize(
  count = n(),
  # Summary stats for Runtime
  avg_rt = mean(total_runtime_mins),
  median_rt = median(total_runtime_mins),
  min_rt = min(total_runtime_mins),
  max_rt = max(total_runtime_mins),

  # Summary stats for Log Views
  avg_views = mean(log_views),
  med_views = median(log_views),
  min_views = min(log_views),
  max_views = max(log_views),

  # Optional: Real views (back-transformed from log)
  typical_views = 10^median(log_views)
) |>
arrange(desc(typical_views)) # Order by performance

# View the table
print(cluster_comparison)

```

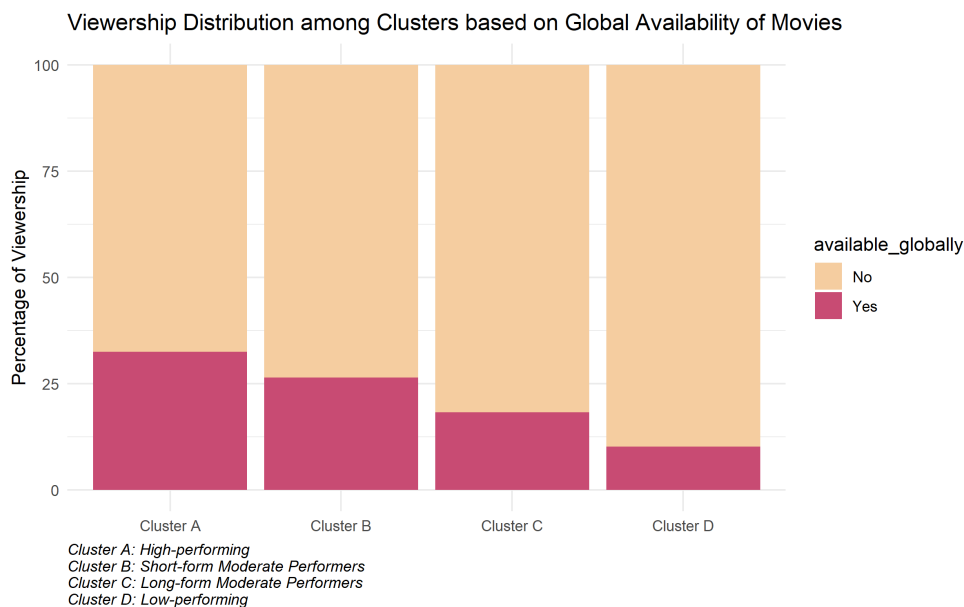
```

# A tibble: 4 × 11
  cluster_names count avg_rt median_rt min_rt max_rt avg_views med_views
<fct>         <int> <dbl>    <dbl> <dbl> <dbl>    <dbl>    <dbl>
1 Cluster A      6018  109.      107    23    201     6.70     6.64
2 Cluster B      8425   93.6       96    20    123     5.85     5.85
3 Cluster C      5032  138.      134   114    209     5.54     5.48
4 Cluster D     16502  101.      101    40    126     5.08     5.00
# i 3 more variables: min_views <dbl>, max_views <dbl>, typical_views <dbl>

```

```
# ===== Visualization 2 =====
# Calculating the statistical summary
summary_stats <- combined |>
  group_by(cluster_names, available_globally) |> # grouping data by per
  cluster and global availability
  summarise(count = n()) |> # count the number of records per cluster and
  global availability
  mutate(percentage = count / sum(count) * 100) |> # calculating the
  percentage
  arrange()

# Plotting a 100% stacked bar chart
summary_stats |> ggplot(aes(x = cluster_names, y = percentage ,fill =
  available_globally)) +
  geom_col(position = "stack") + # stacked bar chart
  labs(
    title = "Viewership Distribution among Clusters based on Global
  Availability of Movies",
    x = NULL,
    y = "Percentage of Viewership",
    caption = "Cluster A: High-performing\nCluster B: Short-form Moderate
  Performers\nCluster C: Long-form Moderate Performers\nCluster D: Low-
  performing"
  ) +
  scale_fill_manual(values = c("#f7cda4", "#ca4f73")) + # Colour mapping
  theme_minimal() +
  # Adding a caption to indicate what each cluster represents
  theme(plot.caption = element_text(hjust = 0, face = "italic", size = 9))
```



```
# Printing the summary statistics  
print(summary_stats)
```

```
# A tibble: 8 × 4  
# Groups:   cluster_names [4]  
  cluster_names available_globally count percentage  
  <fct>         <chr>             <int>      <dbl>  
1 Cluster A     No                4060      67.5  
2 Cluster A     Yes               1958      32.5  
3 Cluster B     No                6195      73.5  
4 Cluster B     Yes               2230      26.5  
5 Cluster C     No                4114      81.8  
6 Cluster C     Yes                918      18.2  
7 Cluster D     No               14820      89.8  
8 Cluster D     Yes               1682      10.2
```

Discussion

PLOT 1:

The K-means clustering algorithm identified four distinct Netflix movie performance segments based on runtime, engagement volume, and viewing efficiency. The scatterplot highlights the 4 distinct regions:

- Cluster A: High-performing
- Cluster B: Short-form Moderate Performers
- Cluster C: Long-form Moderate Performers
- Cluster D: Low-performing

Cluster A represents the highest-performing segment, characterized by both high engagement and high viewing efficiency. This is further supported by the violin and box plots, which show that this cluster has the highest median views and median efficiency among all groups. The average runtime for this cluster is 109 minutes (approximately 1 hour 50 minutes),

Cluster B consists of shorter movies, with an average runtime of 93 minutes (approximately 1 hour 30 minutes). This cluster demonstrates moderate engagement levels while maintaining relatively strong viewing efficiency, especially when compared with longer-form content (Cluster C).

Cluster C movies have a longer runtime, with an average runtime of 138 minutes (approximately 2 hours 20 minutes). The scatterplot shows that viewing efficiency declines as runtime increases. Hence, this cluster too had moderate engagement level and viewing efficiency (lower than Cluster B).

Comparing the two moderately performing groups, Cluster B and Cluster C, the shorter-form movies in Cluster B achieve 700K views, while Cluster C achieve around 300K views. **This suggests that shorter runtimes are generally associated with higher engagement levels among moderately performing films.**

Cluster D represents the lowest-performing segment, with the lowest engagement and viewing efficiency. However, this segment has an average runtime similar to Clusters A and B (average runtime of 105 mins), which are the top 2 performing clusters. **This indicates that runtime alone does not fully explain poor performance.**

The statistical summary further shows that Cluster D contains the largest share of movies, accounting for nearly 46% of the dataset (16,500 out of 35,977 movies). In contrast, only around 6,000 movies (approximately 17%) belong to Cluster A, the highest-performing segment.

Another important observation is that as the runtime of movies varies within each cluster, Cluster A movies manage to stay highly efficient regardless of their length, whereas the

rest of the clusters observe a decrease in efficiency as the movies get longer. We can notice that the runtime increases from Cluster B to D to C and the efficiency decreases from Cluster B to C to D.

This tells us that **runtime is indeed an important contributing factor towards determining the performance and engagement of a movie. However, runtime alone does not determine the performance of a movie.**

PLOT 2:

Cluster A, the highest-performing cluster, has the largest proportion of globally available titles among all four clusters. In fact, 32.5% of movies of Cluster A are globally available compared to 10.2% of globally available movies in Cluster D (lowest performing cluster AND largest cluster among the 4 clusters). Clusters B and C have around 26% and 18.2% of movies available globally, respectively. Hence, **a movie in Cluster A is nearly 3 times more likely to be globally available than a movie in Cluster D.**

As observed in Figure 1, the clusters follow a **clear performance ranking: Cluster A > Cluster B > Cluster C > Cluster D**. This same ordering is reflected in the percentage of globally available titles within each cluster, suggesting a **strong positive relationship between global availability and movie performance.**

This indicates that **global availability plays an important role in driving higher views and stronger engagement.** Movies that are accessible to a wider international audience appear more likely to belong to the higher-performing clusters.

Generative AI Disclosure

Made use of Generative AI ONLY to search for the base syntax of the ggplot functions in R and improve the writing style of the sections (ONLY to be used as a reference).